

Level 4 – Mastery

Duração estimada: 6+ meses (24+ semanas · 5-10h/semana)

Pré-requisito: Nível 3 completo

Foco: Sistemas distribuídos, arquiteturas complexas, liderança técnica, performance extrema

Projeto principal: Plataforma de Streaming de Dados

Skills ao final do nível

- [] **System Design:** CAP theorem, consistência/particionamento, leader election, quorum
 - [] **Event Sourcing & CQRS:** Event store, projeções, sagas coreografadas
 - [] **Kafka:** Tópicos, partições, consumer groups, exactly-once, stream processing
 - [] **Infra as Code:** Terraform – VPC, EC2, RDS, EKS/GKE, state management
 - [] **Cloud:** AWS/GCP – deploy de sistemas distribuídos
 - [] **Segurança:** OWASP Top 10, OAuth2/OIDC, mTLS, secrets management
 - [] **C++ Avançado:** Memory-mapped files, zero-copy, lock-free data structures
 - [] **Distributed Systems:** Raft/Paxos (conceitos), CRDTs, distributed consensus
 - [] **Performance:** Flamegraphs, heap profiling, CPU profiling, benchmarking
-

Semana a Semana

SEMANA 1 – System Design: CAP + PACELC

 **Teoria (5h)** - Leia: **Design Data-Intensive Applications** (Kleppmann) – Caps 1 a 4 - Leia: [System Design Primer \(GitHub\)](#) - Assista: [CAP Theorem Explained](#)

Conceitos chave: - CAP: Consistency, Availability, Partition Tolerance (escolha 2) - PACELC: se Partição → C ou A; senão → L ou C - Consistência: strong, eventual, causal, read-your-writes - Quorum: $R + W > N$ para consistência forte

 **Prática (3h)** - Desenhe 3 sistemas e decida os trade-offs de CAP: 1. Sistema bancário (consistency-critical) 2. Feed de notícias (availability-critical) 3. Chat (eventual consistency aceitável)

SEMANA 2 – System Design: Prática (Twitter, YouTube, Uber)

 **Teoria (3h)** - Leia: **Design de Sistemas** (Alex Xu) – Vol 1 & 2 - Leia: [Designing Data-Intensive Applications] – Caps 5-6 (replicação, particionamento)

 **Prática (5h)** - Projete 3 sistemas completos: 1. Design do Twitter/Threads 2. Design do YouTube 3. Design de um banco NoSQL (mini-DynamoDB)

Para cada:

1. Requirements (funcionais + não funcionais)
2. Estimativas (DAU, tráfego, storage)
3. Data model
4. High-level design (diagrama)
5. Deep dive (componentes críticos)
6. Trade-offs

 **Exercício:** Crie repositório `system-design-practice` com os designs documentados em markdown.

SEMANA 3 – Raft e Consenso Distribuído

 **Teoria (4h)** - Leia: [Raft Paper \(resumo visual\)](#) - Assista: [Raft Explained \(em português\)](#) - Estude: leader election, log replication, safety, membership changes

 **Prática (3h)** - Simule Raft: [Raft Simulator](#) - Implemente um mini-leader-election (conceitual)

SEMANA 4 – Event Sourcing + Event Store

📖 **Teoria (4h)** - Leia: [Martin Fowler – Event Sourcing](#) - Leia: [Event Store in Production](#) - Estude: append-only log, snapshots, projections, idempotência

🔧 **Prática (5h)**

```
// Event Store com PostgreSQL (append-only)
interface Event {
  aggregateId: string;
  aggregateType: string;
  eventType: string;
  data: Record<string, any>;
  version: number;
  timestamp: Date;
}

class PostgresEventStore {
  async append(events: Event[]) {
    const client = await pool.connect();
    try {
      await client.query('BEGIN');
      for (const event of events) {
        // Verificar versão (optimistic concurrency)
        const current = await client.query(
          'SELECT MAX(version) FROM events WHERE aggregate_id = $1',
          [event.aggregateId]
        );
        if (current.rows[0].max >= event.version) {
          throw new ConcurrencyError('Version conflict');
        }
        await client.query(
          `INSERT INTO events (aggregate_id, aggregate_type, event_type, data, version)
          VALUES ($1, $2, $3, $4, $5)`,
          [event.aggregateId, event.aggregateType, event.eventType,
            JSON.stringify(event.data), event.version]
        );
      }
      await client.query('COMMIT');
    } catch (e) {
      await client.query('ROLLBACK');
      throw e;
    }
  }

  async getAggregate<T>(
    aggregateId: string,
    apply: (state: T, event: Event) => T,
    initialState: T
  ): Promise<T> {
    const { rows } = await pool.query(
      'SELECT * FROM events WHERE aggregate_id = $1 ORDER BY version ASC',
      [aggregateId]
    );
    return rows.reduce((state, row) => apply(state, row), initialState);
  }
}
```

SEMANA 5 – CQRS + Sagas Coreografadas (Aprofundamento)

📖 **Teoria (3h)** - Leia: [CQRS \(Martin Fowler\)](#) - Leia: [Saga Pattern \(Microsoft\)](#) - Estude: command side vs query side, projeções materiais, eventual consistency

🔧 **Prática (5h)**

```
// CQRS: Command e Query separados

// Command Side (escreve no Event Store)
class CreateOrderHandler {
  constructor(private eventStore: EventStore) {}
  async handle(command: CreateOrder) {
    await this.eventStore.append([
      {
        aggregateId: command.orderId,
        eventType: 'OrderCreated',
        data: { userId: command.userId, items: command.items, total: command.total },
        version: 1
      }
    ]);
  }
}

// Query Side (lê da projeção)
class OrderQueryHandler {
  constructor(private readDb: PrismaClient) {}
  async getOrder(orderId: string) {
    return this.readDb.orderView.findUnique({ where: { id: orderId } });
  }
}

// Projeção (atualiza view a partir de eventos)
class OrderProjection {
  constructor(private readDb: PrismaClient) {}
  async handle(event: Event) {
    if (event.eventType === 'OrderCreated') {
      await this.readDb.orderView.create({
        data: {
          id: event.aggregateId,
          userId: event.data.userId,
          total: event.data.total,
          status: 'pending'
        }
      });
    }
  }
}
```

SEMANA 6 – Kafka: Streaming de Eventos

📖 **Teoria (4h)** - Leia: [Kafka Documentation – Architecture](#) - Assista: [Hussein Nasser – Kafka Deep Dive](#) - Estude: partições, offsets, consumer groups, ISR, exactly-once semantics

🔧 **Prática (5h)**

```
// Kafka producer
const producer = kafka.producer({ idempotent: true });
await producer.connect();
await producer.send({
  topic: 'events',
  messages: [{ key: orderId, value: JSON.stringify(event), partition: 0 }]
});

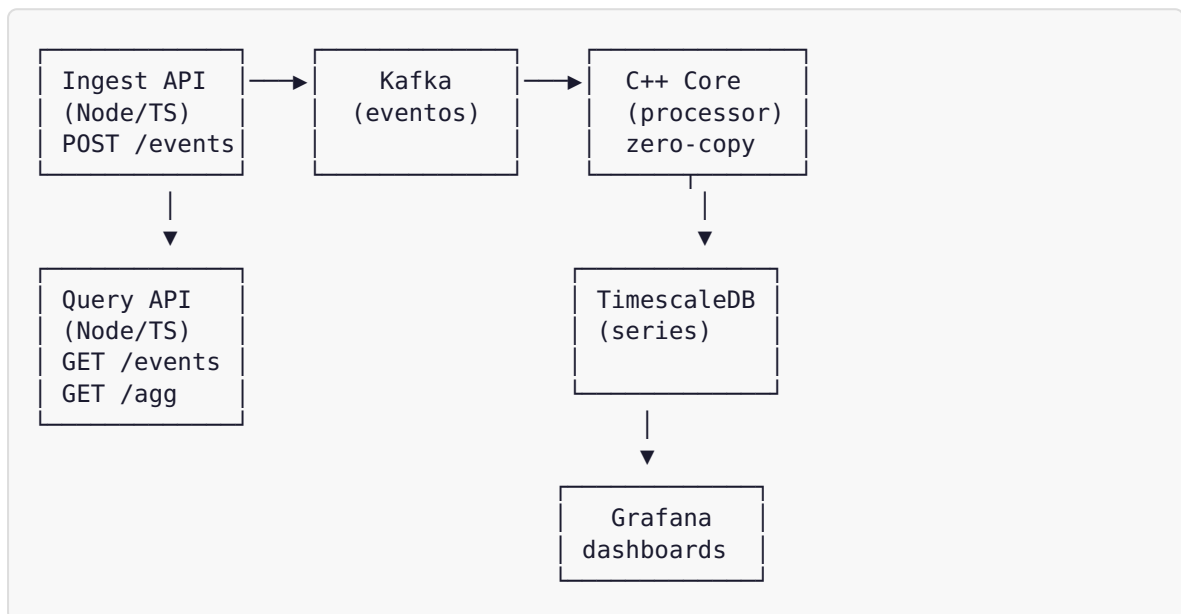
// Kafka consumer
const consumer = kafka.consumer({ groupId: 'payment-service' });
await consumer.connect();
await consumer.subscribe({ topic: 'events', fromBeginning: true });
await consumer.run({
  eachMessage: async ({ topic, partition, message }) => {
    // Processa com exactly-once
    await processEvent(message);
    // Commit manual
    await consumer.commitOffsets([ { topic, partition, offset: message.offset } ]);
  }
});

// Stream processing: counting windowed aggregates
// Kafka Streams ou ksqlDB
```

SEMANAS 7-12 – 🚀 Projeto: Plataforma de Streaming de Dados

Stack: C++ (core), Node/TS (API), Kafka, TimescaleDB, Terraform, K8s

🏗️ **Arquitetura:**



Funcionalidades (Semanas 7-9):

1. **Ingest API (Node/TS):** - Autenticação + rate limiting - Publica eventos no Kafka `typescript POST /api/events { sensorId, metric, value, timestamp }`
2. **C++ Core Processor:** - Consome do Kafka (librdkafka) - Processamento memory-mapped - Agregação por janelas (1min, 5min, 1h)
3. **Query API (Node/TS):** - Time-bucket queries no TimescaleDB `sql SELECT time_bucket('1 hour', ts) AS bucket, AVG(value) FROM sensor data WHERE sensor_id = $1 AND ts BETWEEN $2 AND $3 GROUP BY bucket ORDER BY bucket;`

Terraform (Semanas 10-12):

```
resource "aws_eks_cluster" "streaming" {
  name      = "streaming-platform"
  role_arn = aws_iam_role.eks.arn
  vpc_config { subnet_ids = aws_subnet.private[*].id }
}

resource "aws_msk_cluster" "kafka" {
  cluster_name      = "streaming-kafka"
  kafka_version     = "3.5"
  number_of_broker_nodes = 3
  broker_node_group_info {
    instance_type = "kafka.m5.large"
    client_subnets = aws_subnet.private[*].id
  }
}
```

Checklist: - [] Ingest API publicando no Kafka - [] C++ processor consumindo e agregando - [] Query API consultando TimescaleDB - [] Terraform: VPC + EKS + MSK + RDS - [] K8s deployments (processor, api, etc.) - [] Grafana: throughput, latência, erros - [] Testes de carga (1M+ eventos)

SEMANA 13 – C++ Avançado: Zero-Copy e mmap

 **Teoria (3h)** - Leia: [Linux mmap man page](#) - Leia: [C++ Performance \(CppCon\)](#) - Estude: page cache, cache misses, memory-mapped I/O, direct I/O

 Prática (5h)

```
class LogProcessor {
    int fd_;
    void* mapped_;
    size_t size_;
public:
    LogProcessor(const string& path) {
        fd_ = open(path.c_str(), O_RDONLY);
        size_ = lseek(fd_, 0, SEEK_END);
        // Mapeia arquivo para memória (zero-copy)
        mapped_ = mmap(nullptr, size_, PROT_READ, MAP_PRIVATE, fd_, 0);
    }

    void process() {
        char* data = static_cast<char*>(mapped_);
        // Processa sem syscalls de leitura
        for (size_t i = 0; i < size_; i++) {
            if (data[i] == '\n') { /* parse line */ }
        }
    }

    ~LogProcessor() {
        munmap(mapped_, size_);
        close(fd_);
    }
};

// Benchmark: read() vs mmap() - 1GB file
```

SEMANA 14 – Segurança: OWASP + OAuth2 + mTLS

 **Teoria (4h)** - Leia: [OWASP Top 10 \(2021\)](#) - Leia: [OAuth2 Simplified](#) - Assista: [OAuth2 + OIDC \(Fireship\)](#)

 Prática (4h)

```
// Helmet - headers de segurança
app.use(helmet());

// CORS configurado rigorosamente
app.use(cors({ origin: ['https://app.meudominio.com'] }));

// Rate limiting estrito em auth
app.use('/api/auth', rateLimit({ windowMs: 15 * 60 * 1000, max: 5 }));

// Input validation SEMPRE
const schema = z.object({ email: z.string().email(), password: z.string().min(8) });

// Secrets management (nunca hardcoded)
docker run -d --name vault -p 8200:8200 vault
// Use: process.env, Vault, AWS Secrets Manager

// mTLS entre microserviços (mutual TLS)
```

SEMANA 15 – Lock-Free Data Structures (C++)

📖 **Teoria (3h)** - Leia: [C++ Memory Model \(atomics\)](#) - Leia: [Lock-Free Data Structures \(Boost\)](#)

🔧 **Prática (5h)**

```

// Lock-free queue (Michael-Scott)
template<typename T>
class LockFreeQueue {
    struct Node {
        T value;
        std::atomic<Node*> next;
    };
    std::atomic<Node*> head_;
    std::atomic<Node*> tail_;

public:
    LockFreeQueue() {
        Node* dummy = new Node{};
        head_.store(dummy);
        tail_.store(dummy);
    }

    void push(T value) {
        Node* node = new Node{value, nullptr};
        Node* tail;
        while (true) {
            tail = tail_.load();
            Node* next = tail->next.load();
            if (tail == tail_.load()) {
                if (next == nullptr) {
                    if (tail->next.compare_exchange_weak(next, node)) break;
                } else {
                    tail_.compare_exchange_weak(tail, next);
                }
            }
        }
        tail_.compare_exchange_weak(tail, node);
    }

    bool pop(T& result) {
        Node* head;
        while (true) {
            head = head_.load();
            Node* tail = tail_.load();
            Node* next = head->next.load();
            if (head == head_.load()) {
                if (head == tail) {
                    if (next == nullptr) return false;
                    tail_.compare_exchange_weak(tail, next);
                } else {
                    result = next->value;
                    if (head_.compare_exchange_weak(head, next)) {
                        delete head;
                        return true;
                    }
                }
            }
        }
    }
};

```

SEMANA 16 – Revisão + Prova Nível 4

Prova:

1. **CAP:** Explique com exemplos reais. Por que "CA" não existe?
2. **Event Sourcing:** Vantagens vs desvantagens. Quando NÃO usar?
3. **Kafka:** Como funciona o particionamento e rebalanceamento de consumers?
4. **Terraform:** State management – o que é, por que importante, onde armazenar?
5. **C++:** Explique zero-copy e memory-mapped files. Quando usar?
6. **Segurança:** Cite 3 vulnerabilidades do OWASP Top 10 e suas mitigações
7. **System Design:** Projete um sistema que processa 10M eventos/dia em tempo real

✓ **Passa se:** Responde $\geq 5/7$ + projeto Streaming Platform funcional na nuvem

SEMANAS 17+ – Estudo Contínuo

Papers para ler: - [MapReduce \(Google\)](#) - [BigTable \(Google\)](#) - [DynamoDB \(Amazon\)](#) - [Spanner \(Google\)](#) - [Raft Consensus](#)

Para se manter atualizado: - **Blogs:** Martin Fowler, High Scalability, Netflix Tech Blog, Uber Engineering - **Conferências:** KubeCon, QCon, Strange Loop, CppCon - **LeetCode:** foco em problemas Hard + System Design - **Contribuições open source:** Node.js, KafkaJS, Express, Prisma, RabbitMQ

Resumo do Nível

Domínio	O que você domina
System Design	CAP, consistência, particionamento, design de sistemas em larga escala
Event Sourcing + CQRS	Event store, projeções, commands/queries separados
Kafka	Streaming, partições, exactly-once, processamento
Infra as Code	Terraform, cloud (AWS/GCP), automação
C++ Avançado	Zero-copy, mmap, lock-free, thread pool

Domínio	O que você domina
Distributed Systems	Raft, consenso, CRDTs, distributed transactions
Segurança	OWASP, OAuth2, mTLS, secrets management
Performance	Profiling, benchmarking, otimização

Este é o nível mais alto do roadmap.

A partir daqui, o estudo se torna contínuo e autodirigido.